

Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management

Haoyu Zheng¹ Fangcheng Fu³ Jia Wu⁴ Binhang Yuan⁵ Yongqiang Zhang²
Hao Wang¹ Yuanyuan Zhu¹ Xiao Yan¹ Jiawei Jiang^{1*}

¹ Wuhan University ² Dameng Database ³ Shanghai Jiao Tong University
⁴ Macquarie University ⁵ HKUST

Abstract

LLM-based workflows compose specialized agents to execute complex tasks, and these agents usually share substantial context, allowing KV-Cache reuse to save computation. Existing approaches either manage KV-Cache at agent level and fail to exploit the reuse opportunities within workflows, or manage cache at the workflow level but assume that each workflow calls a static sequence of agents. However, practical workflows are typically dynamic, where the sequence of invoked agents and thus induced cache reuse opportunities depend on the context of each task. To serve such dynamic workflows efficiently, we build a system dubbed PBKV (**P**rediction-**B**ased **KV**-Cache Management). For each workflow, PBKV predicts the agent invocations in several future steps by fusing the guidance from historical workflows and context of the target workflow. Based on the predictions, PBKV estimates the reuse potential of cache entries and keeps the high-potential entries in GPU memory. To be robust to prediction errors, PBKV utilizes the predictions conservatively during both cache eviction and prefetching. Experiments on three workflow benchmarks show that PBKV achieves up to $1.85\times$ speedup over LRU on dynamic workflows, and up to $1.26\times$ speedup over the SOTA baseline KVFlow on the static workflow.

1 Introduction

Large Language Model (LLM)-based multi-agent systems have become a popular paradigm for complex reasoning and automation. Frameworks such as LangChain [1], AutoGen [2], and MetaGPT [3] decompose a complex task into a group of collaborating *agents* (e.g., a Planner, Coder, and Tester in code-generation pipelines), where each agent is invoked as a single LLM request. Such multi-agent collaboration on a single task is known as an *agentic workflow*.

Modern inference engines (e.g., vLLM [4] and SGLang [5]) share Key/Value tensors (i.e., KV-Cache) across requests, turning expensive prefill into cheap cache lookups. In an agentic workflow, agents naturally share substantial context, including the system prompt, tool/agent descriptions, and the history accumulated by upstream agents [6, 1, 7]. We term this reuse pattern *workflow-level cache reuse*. Ideally, the KV-Cache produced by one agent should be retained for the downstream agents, where the cache hit rate can exceed 90% [6]. However, in practice, this potential is bounded by *limited* GPU memory, making effective KV-Cache management crucial in multi-agent serving.

KV-Cache management is fundamentally a problem of *predicting* future access patterns. Existing inference engines commonly adopt the Least Recently Used (LRU) policy, which rests on a simple heuristic that the longest-idle cache is unlikely to be reused. However, in multi-agent settings, cache reuse is determined by the *workflow’s structure* rather than temporal locality. A cache entry may remain idle across multiple agent invocations (e.g., during tool calls or transitions between agents) before being reused, and LRU may evict it prematurely during this idle period. As a result, LRU’s temporal-locality assumption is misaligned with the structural nature of workflow-level cache reuse.

*Corresponding author: jiawei.jiang@whu.edu.cn

An idealized alternative is the classic Belady’s algorithm [8], which achieves *offline*-optimal cache eviction by evicting the cache whose next access is farthest in the future. The recent work KVFlow [9] adapts this idea to multi-agent settings by assuming a *predefined* and *static* agent step graph (i.e., a global DAG that specifies the invocation orders of all agents), and evicting the KV-Cache of the agent whose next invocation is farthest away (i.e., with the largest ‘steps-to-execution’). This design rests on a strong assumption: the future invocation order of agents is *known a priori*. This assumption fails on realistic dynamic workloads, where workflows are *runtime-dependent*. For example, a Tester may send the code back for rewriting; and a Retriever may spawn new sub-queries based on intermediate results. The realistic workflow structure cannot be determined in advance, thus effective KV-Cache management must **shift from assuming the future to predicting it**.

Challenge. However, realizing this shift faces two conflicting challenges: (i) agentic workloads are inherently hard to predict [10–14], because the stochastic decoding of LLMs injects uncertainty at each step, which propagates along the workflow path and yields high variability in the resulting agent invocation trace. Industrial studies from Microsoft [15] and IBM [16, 17] report that such uncertainty is common and consequential in agentic systems. (ii) Prediction errors are disproportionately costly. The cost of mistakes is amplified by the workflow: a cache miss in multi-agent serving may force re-prefill of tens of thousands of tokens *accumulated* along the entire workflow context.

Taken together, the structure of agentic workloads exposes substantial reuse potential, yet inevitably entails prediction errors and the steep cost of erroneous evictions. This motivates the central question of our work: *in a dynamic multi-agent system, how can we design a KV-Cache management framework that (i) is robust to predictor quality and (ii) benefits continuously even from imperfect predictors?*

Our Solution. Our key insights: (i) *multi-step* prediction with *complementary* signals can effectively replace the static assumption; and (ii) *conservative* designs can yield *stable* and *robust* gains.

Specifically, **for the predictor**, we propose two design principles: (i) *Complementary signal fusion*, which combines *cross-workflow* agent transition patterns with *per-request* semantics; (ii) *Multi-step horizon*, which is motivated by the fact that cache reuse distances in agentic workflows can span several invocations (e.g., across a retry loop), so a single-step predictor cannot distinguish cache that will be reused later from useless cache. Multi-step prediction can prevent this myopia *without* assuming a static workflow. **For the cache manager**, we likewise propose two designs: (i) *Hierarchical eviction*. Guided by the observation that the private cache of a terminated workflow has negligible reuse potential regardless of any prediction, we reclaim such *retired* cache first and then rank the remaining active cache by a lookahead reuse score from the predictor, yielding *stable* gains in the common case and *graceful* degradation under poor predictions. (ii) *Conservative prefetching*, which carefully *trades off* the cost and benefit of prefetching and consumes *only* otherwise-idle GPU space and PCIe bandwidth, so that it does not backfire even under poor predictions.

Together, these designs form a **Prediction-Based KV-Cache** management framework, which we call PBKV. We evaluate PBKV on realistic dynamic workflows, where it reduces the average workflow latency by up to $1.85\times$ and improves the KV-Cache hit rate by up to $2.55\times$ over LRU. On static workflows, PBKV outperforms KVFlow on these metrics by up to $1.26\times$ and $1.39\times$, respectively. Furthermore, PBKV’s predictor is *pluggable*; to ensure *robustness* across predictors, we prove that the performance is *Lipschitz-continuous* in prediction error, establishing *graceful degradation*.

Contributions. In summary, we make the following contributions:

- We formalize KV-Cache management in multi-agent serving as a problem of *predicting* future access patterns, and provide design guidelines for predictors in this setting.
- We propose PBKV, guided by two design principles, i.e., *hierarchical eviction* and *conservative prefetching*, that together identify cache with low reuse value for reclamation while preserving high-value cache. We further provide a *Lipschitz guarantee* that PBKV’s performance degrades *gracefully* even under prediction error.
- We implement and evaluate PBKV across diverse workloads and LLMs. PBKV consistently outperforms LRU and the SOTA baseline in both cache hit rate and end-to-end performance.

2 Preliminaries

LLM-based Multi-Agent Serving. A multi-agent application can be abstracted as a directed graph $G = (V, E)$, where each node in V corresponds to an agent and each edge in E denotes an admissible transition between agents. Since G enumerates *every possible* transition pattern, we refer to it as the

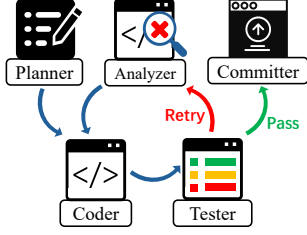


Figure 1: A call graph for the code-generation task. The Tester conditionally triggers a retry path through Analyzer and Coder, i.e., a retry loop.

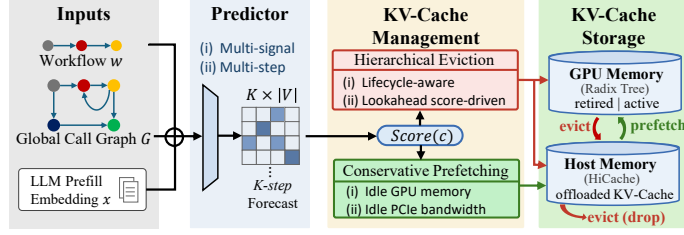


Figure 2: **System overview of PBKV.** For each active workflow w , the predictor produces a K -step forecast over upcoming agent invocations. The forecast drives a shared scoring function $Score(c)$, which feeds both a hierarchical eviction policy and a conservative prefetching policy on the two-tier KV-Cache storage.

global call graph. Figure 1 shows a representative example for a code-generation task. Notably, G can contain loops, capturing retry loops common in realistic agentic workflows. A **workflow** is a single execution instance on G , consisting of an ordered sequence of agent invocations $a_1, a_2, \dots$, where each $a_i \in V$ and each consecutive pair $(a_i, a_{i+1}) \in E$. Each **agent invocation** is an LLM request with a specific prompt, and thus benefits from KV-Cache reuse to reduce latency.

Agents typically share a large amount of KV-Cache, which we classify into two categories: (i) *global cache*, shared across agent instances from *different* workflows (e.g., the system prompt, tool/agent descriptions, and knowledge documents); and (ii) *private cache*, produced by upstream agents of a particular workflow and reusable only by downstream agents of the *same* workflow. Cross-workflow reuse of private cache is generically infeasible, because even under identical user prompts, the stochastic decoding of LLMs makes the workflows diverge within a few tokens.

Radix Tree. PBKV is built on *SGLang* [5], which organizes prefix-shared cache as a *Radix Tree*, with each node holding a contiguous token segment reusable by all requests sharing that prefix. *HiCache* [18] extends the cache into a two-tier hierarchy where cache evicted from GPU is retained in host memory and swapped back on a hit, avoiding full re-prefill at the cost of a PCIe transfer.

3 System Overview

The underlying design principle of PBKV is: predicting upcoming agent invocations to evaluate the *reuse value* of existing KV-Cache, which then drives both KV-Cache eviction and prefetching. As shown in Figure 2, PBKV consists of three components: (i) a *predictor* that produces a multi-step forecast for each active workflow, (ii) a set of *KV-Cache management policies* that translate the forecasts into eviction and prefetching decisions, and (iii) a *two-tier KV-Cache storage* organized as a Radix Tree on GPU memory and HiCache on host memory, on which the policies operate.

The predictor serves as the foundation of PBKV. **First**, it fuses two complementary signals: (i) *graph-level* agent transition patterns *shared* across requests, encoded in the global call graph G ; and (ii) *workflow-level specifics* of the current request, reused from the LLM prefill embedding x . **Second**, it emits K probability distributions over upcoming agent invocations, rather than only the next one, preventing *myopic eviction*. For instance, in Figure 1, when the current agent is Analyzer, a single-step predictor would reveal only the next Coder invocation and miss the Tester re-invocation two steps later, risking a wrongful eviction of the Tester’s cache and a costly re-prefill.

The KV-Cache management policies consist of an eviction policy and a prefetching policy. They share a common reuse scoring mechanism (i.e., multi-step lookahead and cross-workflow aggregation) and a common design philosophy (i.e., embedding deterministic guardrails within a probabilistic system). Specifically, (i) *hierarchical eviction* reclaims retired cache from terminated workflows first as it carries no reuse potential, and only after it is exhausted does score-driven eviction take over the active cache. Thus, performance degrades gracefully when the predictor is unreliable, while the upside is preserved when it is accurate. (ii) *Conservative prefetching* is motivated by the asymmetry that a prefetch always pays its cost while the benefit materializes only when the prediction is correct. PBKV therefore restricts prefetching to otherwise-idle GPU space and PCIe bandwidth, so that even under poor predictions, prefetching neither displaces valuable cache nor competes for on-path bandwidth. In the following section, we elaborate on the design of each component and their coordination.

4 Design of PBKV

In this part, we design a predictor as the foundation of subsequent KV-Cache Management.

4.1 Workflow Prediction

Predictor Design. Building on the principles above (i.e., multi-signal fusion and multi-step horizon), we instantiate the predictor with two desiderata: (i) leveraging structural priors in the global call graph G , and (ii) generalizing to unseen workflow prefixes at runtime to accommodate the dynamic nature of agentic workflows. We adopt **GraphSAGE** [19] as the backbone, which satisfies both: (i) it operates directly on the *graph* through neighborhood sampling and aggregation, preserving the structural priors, and (ii) it is *inductive*, performing forward inference on unseen prefixes.

As shown in Figure 3, the predictor fuses three complementary streams. (i) *Topology-aware agent embedding* h_{cur} : each agent’s learnable embedding is refined by two GraphSAGE layers over a transition matrix estimated from offline traces, so that the resulting $H^{(2)}$ encodes both the agent’s identity and its neighborhood in G . (ii) *Attention-based history aggregation* h_{path} : since two workflows at the same current agent may have arrived through very different prefixes, we summarize prior agent representations via scaled dot-product attention with h_{cur} as the query, up-weighting the most informative prefix agents over naive mean pooling. (iii) *Semantic signal from*

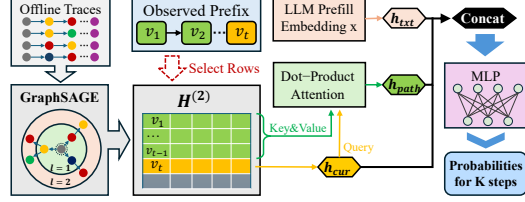


Figure 3: **Architecture of the predictor.** It fuses a topology-aware agent embedding from GraphSAGE (h_{cur}), an attention-based workflow prefix summary (h_{path}), and a semantic signal reused from prefill (h_{txt}), then jointly predicts the next K agent probability distributions via an MLP.

prefill h_{txt} : to distinguish requests with the same prefix but diverging intent, we project the post-norm hidden state of the last prefill token, obtaining the signal essentially for free. The three streams are concatenated and passed through a two-layer MLP that jointly emits logits over agents for each of the next K steps in a single forward pass, avoiding autoregressive error accumulation while keeping per-invocation latency to that of a single inference. Full architecture is detailed in Appendix B.

Training and Performance. The predictor is trained on offline invocation traces with cross-entropy loss, pairing each prefix and its prefill embedding with the next K agents as labels (padding positions after $\langle \text{END} \rangle$ are masked). The resulting predictor has roughly 350K parameters and processes a batch of 1,024 requests in 1.56 ms, rendering its runtime overhead negligible. On the HoVer [20] dataset with the LangChain [1] framework, training on 1K traces achieves an accuracy of 0.94 at 1-step and 0.77 at 3-step (500-trace test set). Scaling curves and comparisons against alternative predictors are provided in Appendix C, with full hyperparameters in the supplementary code.

4.2 Lookahead KV-Cache Eviction

Building on the multi-step predictor, we design a *lookahead* KV-Cache eviction policy.

4.2.1 Base: Lifecycle-Aware KV-Cache Eviction

Our observation and design. Once a workflow terminates, its residual private cache (which we call *retired cache*) has negligible reuse potential and should be reclaimed first. Under LRU, such cache can only age out passively, and LRU may evict still-valuable cache of other workflows during this aging window. Additionally, retired cache is not all equivalent, we further rank them by the number of workflows that have accessed them. Thus, the popular shared prefix can be preserved longer.

Implementation. We tag each cache node with the workflows that have accessed it. The server continuously listens for and records workflow termination messages from clients. Any cache node whose associated workflows have *all* terminated is tagged as retired and prioritized for eviction. Despite its simplicity, this change alone improves the average hit rate by up to $1.66\times$ in our experiments.

4.2.2 Lookahead Score-Driven KV-Cache Eviction

Retired cache is not always abundant. Under high load, once the retired cache is drained, lifecycle-aware eviction degenerates back to LRU. We notice that the reuse likelihood also *varies* across the active cache. For example, (i) among the *global cache*, descriptions of popular agents are reused more frequently than those of rarely used ones; and (ii) among the *private cache*, retention priority should

scale with the predicted probability that the owning agent is invoked in the future. We therefore extend eviction from a binary lifecycle label to a continuous score that reflects predicted reuse value.

Cross-Workflow Value Aggregation. KV-Cache is shared across workflows, thus its reuse value should be aggregated *globally*. Therefore, the score of a cache node should grow with (i) the number of workflows likely to reuse it, and (ii) each workflow’s reuse probability. We further annotate each cache node c with a per-workflow access indicator vector $A_w(c) \in \{0, 1\}^{|V|}$. For example, if c has been accessed by Agents 1 and 3 from workflow w , then $A_w(c) = [1, 0, 1, \dots]$. For each workflow w , the predictor emits a next-step distribution, which we decompose into an agent-access probability vector P_w and a termination probability $p_{w, \langle \text{END} \rangle}$. The single-step reuse value of c is then defined as

$$\text{Value}(c) = \sum_{w \in \mathcal{W}^{\text{act}}(c)} A_w(c) \cdot P_w \quad (1)$$

where $\mathcal{W}^{\text{act}}(c)$ denotes the set of active workflows associated with node c . Intuitively, $\text{Value}(c)$ aggregates the probability that each workflow’s next invocation will touch node c . Notably, this cross-workflow aggregation *naturally* protects the global cache and popular-prefix cache.

From One-Step to K -Step Lookahead. As described above, a single-step view remains myopic, so we extend the score to a K -step horizon by leveraging the predictor’s multi-step outputs. Two practical effects must be accounted for: (i) predictions further into the future are objectively less reliable, regardless of the predictor’s stated confidence; and (ii) steps following a predicted termination are meaningless and would pollute the score if left in. To address them, we introduce a confidence decay factor $\gamma < 1$ and a *cumulative survival probability* $s_w^{(k)} = \prod_{j=1}^{k-1} (1 - p_{w, \langle \text{END} \rangle}^{(j)})$, i.e., the probability that workflow w remains active at step k . Figure 4 illustrates the computation, which we formalize as:

$$\text{Score}(c) = \sum_{k=1}^K \gamma^{k-1} \sum_{w \in \mathcal{W}^{\text{act}}(c)} s_w^{(k)} \cdot A_w(c) \cdot P_w^{(k)} \quad (2)$$

Implementation. On every workflow state change (i.e., new agent invocation or termination), the predictor refreshes its forecast and updates the scores of the affected cache nodes. On eviction, nodes are sorted in ascending order of $\text{Score}(c)$ and evicted until the requested space is freed.

4.2.3 Hierarchical Eviction: Unifying the Two Policies

The two policies can be naturally combined because retired cache nodes receive a score of 0 under Equation 2 (as they satisfy $\mathcal{W}^{\text{act}}(c) = \emptyset$). That said, the “no value” judgment of retired cache is *deterministic*, whereas the “zero score” of an active node is merely a *probabilistic* estimate bounded by both the horizon K and the predictor’s accuracy. We therefore adopt a *hierarchical eviction* strategy, in which active cache is spared until all retired cache is drained. The underlying design philosophy is to *embed deterministic guardrails within a probabilistic system*, so that performance degrades gracefully under unreliable predictions while preserving the upside under good predictions.

4.3 Conservative KV-Cache Prefetching

Building on the predictor, a complementary optimization is to proactively load likely-to-be-reused cache nodes into GPU memory before they are hit, leveraging SGLang’s HiCache infrastructure to hide transfer latency behind ongoing decode steps rather than exposing it on the request critical path.

Prefetching Principle. At its core, prefetching trades *known-valuable* GPU cache for *speculatively valuable* host cache, with the *fixed costs* in scheduling and PCIe bandwidth. Given the inevitable prediction errors in dynamic workflows, the asymmetry between deterministic cost and probabilistic benefit motivates a *conservative* principle that prefers risk avoidance over aggressive speculation.

Prefetching Design. Concretely, we do *not* evict any active cache to make room for prefetched data, for three considerations: (i) as argued above, evicting known-valuable cache is inherently risky under dynamic workflows; (ii) under high concurrency, such evictions can disrupt the radix tree’s prefix structure; and (iii) if the evicted active cache is hit shortly after, the system pays an additional GPU-reload cost or, in the worst case, a full re-prefill. *Instead*, we restrict the prefetch region to

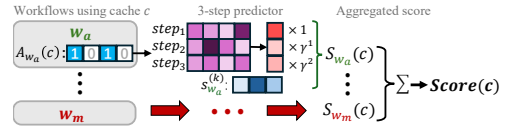


Figure 4: **Computing the cross-workflow reuse score.** For each active workflow w accessing cache node c , the K -step ($K=3$ here) predictor outputs per-step access probabilities, which are weighted by the survival probability $s_w^{(k)}$ and confidence factor γ^{k-1} and summed across m workflows as $\text{Score}(c)$.

the union of currently *free* space and *retired* cache (§4.2.1), whose total size we denote as S_a . This design guarantees that prefetching perturbs existing GPU-resident cache only minimally, if at all.

Beyond GPU space, prefetching also consumes PCIe bandwidth and therefore *competes* with prefill requests. To avoid interference, PBKV activates prefetching only on pure-decode batches (>90% of all batches in our measurements). Within such a batch, we bound the transferable volume as $S_{bw} = \text{Bandwidth} \cdot \text{StepDuration}$, where the PCIe bandwidth and decode step duration are dynamically updated by their runtime averages. The final budget is $S = \min\{S_a, S_{bw}\}$, which typically hides prefetching cost behind a single decode step and uses only otherwise-idle bandwidth.

To rank candidate nodes in the host memory, we reuse the one-step reuse value $\text{Value}(c)$ defined in Eq. 1. We deliberately restrict ranking to the *one-step* horizon because prefetching is naturally incremental: a node that will be reused two steps ahead can simply be fetched at the next step. In contrast, eviction must look ahead across K steps, since a mistaken eviction incurs a costly reload or even a full re-prefill. Selecting the optimal subset under budget S is formally a knapsack problem and hence *NP-hard*. Since prefetch decisions lie on the scheduler’s critical path, we use a lightweight greedy heuristic that loads candidate nodes in descending order of value until the budget is exhausted.

An aggressive variant that permits evicting active cache for prefetch space is evaluated in Appendix E.

5 Discussion

Positioning. While our primary contribution is KV-cache management, we additionally provide design guidelines for predictors in this setting (Section 7 explains why existing predictors fall short) and instantiate one accordingly. PBKV deliberately keeps the predictor module *pluggable*, so that it can benefit from future predictors. We further prove in Appendix G that PBKV degrades *gracefully* even under poor predictions (i.e., the degradation is *Lipschitz-continuous*), that is:

Theorem 5.1. *Let $\hat{E}_B$ and E_B^* be PBKV’s eviction set and the cost-minimizing set under the ground truth, respectively, and ϵ_n^γ the per-node prediction error. The eviction cost regret of PBKV satisfies*

$$0 \leq \mathcal{R}(B) := \mathcal{L}(\hat{E}_B) - \mathcal{L}(E_B^*) \leq \frac{1}{2(1-\gamma)} \sum_{c \in \hat{E}_B \triangle E_B^*} \epsilon_c^\gamma,$$

where $\hat{E}_B \triangle E_B^*$ denotes the symmetric difference between PBKV’s eviction set and the optimum. The bound $\mathcal{R}(B) \rightarrow 0$ when the prediction is perfect and grows linearly in the prediction error.

Complexity. PBKV maintains the score of candidate nodes in a *heap* structure. Inspecting the top candidate takes $O(1)$, while popping, insertion, and score updates take $O(\log n)$ for n cache nodes, keeping the scheduler overhead negligible even under high concurrency (measured in Appendix F).

Limitations. (i) The predictor needs to be trained on a *specific* workload. (ii) Cache-reuse patterns across agents are not universal but rather depend on the *client’s specific* message-passing convention. The indicator $A_w(c)$ in Eq. 1 captures a *basic form*, namely the set of agents that have directly accessed c in workflow w . It admits straightforward *customization*, e.g., when a framework specifies that agent j inherits cache accessed by agent i , $A_w(c)[j]$ can be set to 1 whenever $A_w(c)[i] = 1$.

6 Experimental Evaluation

In this section, we evaluate PBKV on realistic multi-agent workloads and ablate each component.

6.1 Experimental Settings

Testbed and Models. Our experiments are conducted on a server with $8 \times$ NVIDIA A6000 (48 GB) GPUs interconnected via NVLink, 128 virtual CPU cores, 512 GB of memory, and 20 GB/s PCIe bandwidth. We use Qwen3-14B and Qwen3-32B (two-way tensor parallelism) as the base LLMs.

Baselines. We compare PBKV against (i) **LRU** on SGLang+HiCache (HICACHE_RATIO=1, allocating equal capacity on GPU and host memory), the default policy of SGLang. (ii) **KVFlow** [9], the SOTA workflow-aware policy, whose eviction is *driven* by the ‘steps-to-execution’ distance on a static DAG. Since this distance is *undefined* under runtime-dependent *loops*, we construct a static workflow scenario for fair comparison. To isolate the contribution of each component, we further evaluate two ablation variants of PBKV: (i) **PBKV-LAE**, which enables only Lifecycle-Aware Eviction; and (ii) **PBKV-HE**, which adds Hierarchical Eviction on top of PBKV-LAE but disables prefetching.

Workloads. We evaluate PBKV on three representative multi-agent workloads, each pairing a public benchmark with a widely-adopted agent framework to reflect *real-world* deployment: (i) *fact verification*, using the HoVer [20] dataset with the LangChain [1] agent framework; (ii) *code*

Table 1: Performance of policies across workloads and LLMs. The metrics are described above. As different workloads have different GPU memory footprints, we set the concurrency limit to 72/24/48, respectively, and study its sensitivity in Section 6.3. The *static* workload is designed for comparison with KVFlow, which assumes a predefined static call graph. (Full results with std see Appendix A)

Workload	Policy	Qwen3-32B			Qwen3-14B		
		Lat. (s)↓	PA. TTFT (s)↓	Hit Rate (%)↑	Lat. (s)↓	PA. TTFT (s)↓	Hit Rate (%)↑
HoVer + LangChain (with iterative refinement <i>loops</i>)	LRU	189.66	16.65	27.09	139.59	10.90	28.79
	PBKV-LAE	146.67	11.83	44.91	119.13	9.57	41.18
	PBKV-HE	108.86	8.95	66.01	80.62	6.53	64.15
	Full PBKV	102.60	8.22	69.10	76.15	5.50	68.29
SWE-bench + AutoGen (with retry <i>loops</i>)	LRU	271.90	5.04	46.34	214.74	4.18	48.53
	PBKV-LAE	233.86	3.73	59.51	177.50	3.26	60.41
	PBKV-HE	177.22	2.46	75.31	141.53	2.19	73.36
	Full PBKV	160.18	2.27	79.94	118.61	2.05	77.07
FinanceBench + CrewAI (<i>static</i>)	LRU	130.34	12.93	27.95	96.11	9.02	26.58
	KVFlow	101.57	9.91	39.87	80.13	7.42	39.65
	Full PBKV	80.53	8.00	53.44	65.01	6.06	55.07

generation, using SWE-bench [21] with Microsoft AutoGen [2]; and (iii) *document analysis*, using FinanceBench [22] with CrewAI [23], which constitutes a *static* workflow for direct comparison against KVFlow. Following prior work [9, 24], we set the number of concurrent workflows to induce memory pressure, thereby effectively evaluating KV-Cache management policies.

Metrics. We evaluate each policy along three *complementary* dimensions: (i) *End-to-end workflow latency*, measured from workflow submission to final completion; (ii) *Per-agent TTFT*, defined as the mean time-to-first-token across every agent invocation along a workflow; (iii) *Average KV-Cache hit rate* (token-level) on GPU memory, which *directly* reflects the quality of cache management.

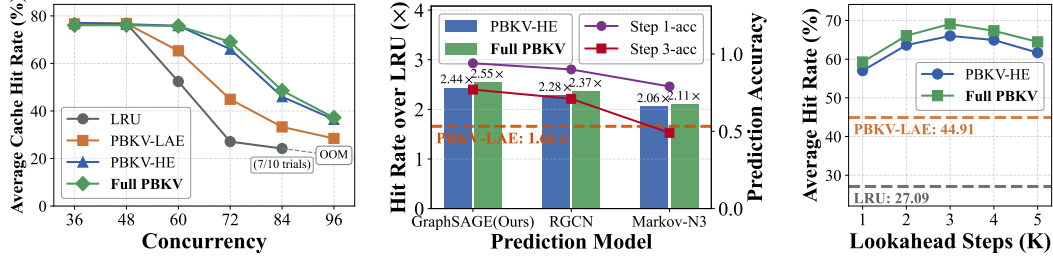
6.2 Main Results.

Observations. We take HoVer + LangChain workload as a representative case. Under $K=3$ and $\gamma=0.7$, Table 1 shows that PBKV reduces end-to-end latency and per-agent TTFT by up to $1.85\times$ and $2.03\times$ over LRU, respectively. This directly confirms that PBKV delivers substantial gains in both system efficiency and quality of service (i.e., QoS). The KV-Cache hit rate exposes the source of these gains: (i) **LRU** achieves a hit rate below 30% on both models, confirming that it fails to capture the workflow-level reuse. (ii) **PBKV-LAE**, which merely identifies and evicts *retired cache* first, already lifts the hit rate by up to $1.66\times$ over LRU, showing that even simple *lifecycle-awareness* is highly effective. (iii) **PBKV-HE** additionally incorporates the workflow predictor and reuse scoring, pushing the hit rate to 66.01%, because it assesses the reuse value of active cache through *cross-workflow aggregation* and *multi-step lookahead*. As a result, eviction decisions remain informed even after the retired cache is exhausted, and the policy never degenerates back to LRU. (iv) **Full PBKV** introduces *speculative prefetching* on top of PBKV-HE and attains a hit rate of 69.10% ($2.55\times$ over LRU).

We further note that the hit rates are higher near the start and the end of each run, when the pressure is low. Restricted to the steady-state interior, LRU’s hit rate drops to $\sim 10\%$ while full PBKV’s sustains $\sim 50\%$. Their behavior is detailed in Section 6.4. Results of other tests exhibit similar trends.

Ablation Study. Comparing the three PBKV variants, the contributions of Lifecycle-Aware Eviction, Hierarchical Eviction, and speculative prefetching are *incremental*. The step from PBKV-LAE to PBKV-HE is pronounced, whereas the step from PBKV-HE to full PBKV is *modest*. This modest gap stems from two factors: (i) the prediction errors of the multi-step predictor on realistic dynamic workflows, and (ii) our deliberately *conservative* prefetching design (Section 4.3), which refuses to gamble known-valuable active cache for speculative gains. Appendix E evaluates an aggressive alternative that evicts active cache to free prefetch space, confirming that our conservative variant delivers stable benefits while avoiding the backfire of aggressive prefetching under poor predictions.

Performance on Static Workflow. Table 1 also reports the results under a static workflow (FinanceBench + CrewAI). KVFlow infers the steps-to-execution from the predefined call graph and prioritizes the cache accordingly, yielding a moderate improvement over LRU. PBKV achieves a *further* gain, which we attribute to two design choices. (i) *Lifecycle-aware eviction*. Like LRU, KVFlow can also mistakenly evict active cache that will still be reused, whereas PBKV’s lifecycle-aware eviction prevents this misjudgment. (ii) *Cross-workflow reuse aggregation*. KVFlow prioritizes each cache node by the *minimum* steps-to-execution across all related workflows, which captures *when* it



(a) Average cache hit rate vs. concurrency. Excessive concurrency causes LRU to run out of memory (OOM). (b) Prediction accuracy of different backbones (lines) and the corresponding hit-rate ratio of PBKV over LRU (bars). (c) Average cache hit rate vs. lookahead horizon K (i.e., prediction step).

Figure 5: Sensitivity analysis of PBKV and its variants on HoVer + LangChain with Qwen3-32B.

will be reused but not how *popular* it is. PBKV instead *aggregates* the predicted reuse contribution from every active workflow accessing the node, so the score of popular cache grows naturally.

Remark on Static Workflows. Given the deterministic agent invocation order in *static* workflows, PBKV could be further tuned by safely relaxing several mechanisms (e.g., adopting more aggressive prefetching and disabling the multi-step confidence decay γ). However, we retain these mechanisms in our evaluation for compatibility and consistency across both dynamic and static workloads.

6.3 Sensitivity Analysis

In this section, we vary several configurations to characterize their impact on PBKV and its variants. All experiments below are conducted on the HoVer + LangChain workload with Qwen3-32B.

Concurrency. It directly governs GPU memory pressure. As shown in Figure 5a, all hit rates degrade as concurrency rises, yet PBKV consistently and greatly outperforms LRU. Below 60 workflows, evictions are too rare to differentiate any policy. At the other extreme, LRU exhausts memory and crashes in 3/10 trials at concurrency 84 and 10/10 at 96. We therefore adopt a concurrency of 72 for HoVer + LangChain in the main experiments to evaluate all policies under reasonable pressure.

Predictor Backbone. PBKV treats the predictor as a pluggable component. We replace our GraphSAGE backbone with R-GCN and a third-order Markov model (Markov-N3), each trained on the same $1K$ traces. As shown in Figure 5b, the variants exhibit lower prediction accuracy, which translates monotonically into lower cache hit rates. Even so, the weakest backbone still strictly outperforms the prediction-free PBKV-LAE, confirming that PBKV is robust to predictor quality and that a reasonable predictor typically delivers additional benefits on top of lifecycle awareness.

Lookahead Horizon. As shown in Figure 5c, the best hit rate occurs at $K=3$: a smaller horizon induces myopic eviction, whereas a larger one dilutes the score with inaccurate long-range predictions. The gap between full PBKV and PBKV-HE stays nearly constant across K by design, since prefetching ranks candidates by the one-step value alone (Section 4.3) and is decoupled from K .

Additional Studies. Due to space constraints, we defer the following studies to the appendix: (i) the accuracy of more predictor variants and how training scale affects it (Appendix C); (ii) sensitivity to the confidence decay coefficient γ (Appendix D); (iii) aggressive vs. conservative prefetching across varying prediction accuracies (Appendix E); (iv) scheduling overhead (Appendix F); and (v) a theoretical analysis of PBKV’s graceful degradation under poor prediction (Appendix G).

6.4 Why PBKV Works: Analysis of Cache Behavior.

To understand the sources of PBKV’s gains, we trace the KV-Cache hit rate of each policy throughout a full run in Figure 6. Key events (marked by dashed vertical lines) divide the run into several phases: (i) **Warm-up (0-30 s).** With ample GPU memory, all policies rapidly exceed 80% without eviction, confirming our motivation in Section 1 that multi-agent workflows expose abundant reuse potential. (ii) **Onset of memory pressure (30-55 s).** Around 30 s, GPU memory saturates and forces evictions, causing LRU’s hit rate (red) to drop sharply. PBKV-LAE (orange) follows the same trajectory, as no workflow has terminated to release retired cache. In contrast, PBKV-HE (blue) and full PBKV (green) decline only *gradually*, since their scoring identifies low-value *active* cache for eviction. (iii) **Retired-cache window (55-80 s).** From 55 s onward, the first-completed workflows release retired cache. PBKV-LAE immediately capitalizes on this signal and recovers visibly. PBKV-HE and PBKV enjoy the same benefit through their hierarchical eviction. Thereafter, LRU stays around a 10% hit rate and only recovers near the end when the draining request pool relieves pressure.

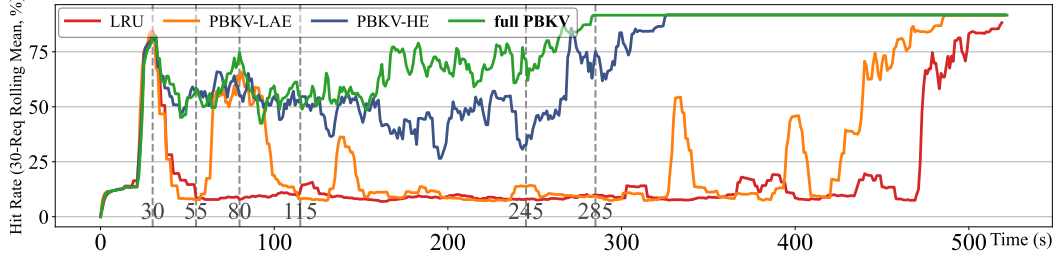


Figure 6: KV-Cache hit rate of each policy over time on the HoVer + LangChain workload, served by Qwen3-14B under a concurrency limit of 72. Dashed vertical lines mark key events.

(iv) **Retired-cache drained (80-115 s).** Once the retired cache is drained, PBKV-LAE reverts to LRU. Thereafter, its hit rate recovers only *briefly* when batches of workflows release retired cache concurrently. PBKV-HE and PBKV also decline but remain well above LRU, thanks to their scoring. (v) **Steady-state serving (115-245 s).** PBKV and PBKV-HE greatly outperform others, showing the lookahead score remains effective under prolonged pressure. PBKV further improves PBKV-HE by a *modest but consistent* margin, reflecting the bounded but stable benefit of *conservative* prefetching. (vi) **Tail phase (after 245 s).** As the request pool drains, eviction pressure subsides and the hit rate climbs back to roughly 90% by completion (around 285 s). Full PBKV enters and exits this phase *first*, achieving the lowest average job completion time and the highest throughput among all policies.

Summary. This phase-by-phase analysis reveals the *complementary* roles of PBKV’s three mechanisms. Lifecycle-aware eviction provides a deterministic improvement whenever retired cache is available; hierarchical eviction sustains improvements under prolonged pressure via multi-step lookahead and cross-workflow aggregation; and speculative prefetching contributes a bounded but stable improvement on top. Together, they enable PBKV to maintain a substantial and sustained lead over LRU, directly translating into lower latency (as shown in Table 1) and higher throughput.

7 Related Work

KV-Cache Management for Multi-Agent Serving. Modern inference engines (e.g., vLLM [4] and SGLang [5]) manage KV-Cache via paged attention and radix-tree prefix caching under a default LRU policy. Orthogonal efforts such as HiCache [18] and LMCache [25] extend this with hierarchical GPU-CPU-disk offloading, while KVCOMM [26] and CacheBlend [27] target non-prefix reuse. All of them are oblivious to workflows. Some recent systems try to close this gap, Continuum [28] targets ReAct-style single-agent tool calling, pinning KV entries with a dynamic TTL learned from per-tool duration statistics, whereas KVFlow [9] assumes a predefined Agent Step Graph and uses ‘steps-to-execution’ scores for eviction and prefetching. They are not well-suited to realistic multi-agent workflows, which are typically dynamic and involve runtime-dependent loops. PBKV fills this gap by driving *workflow-level* KV-Cache management with multi-step *prediction* for each workflow.

Agentic Workflow Prediction. A growing body of work predicts the structure of agentic workflows. Parrot [29] introduces semantic variables to recover inter-request dependencies, Ayo [30] compiles LLM applications into primitive-level dataflow graphs for end-to-end optimization, and Autellix [10] elevates programs to first-class scheduling units. Such abstractions assume a (*near*-)static DAG or dataflow graph and struggle to faithfully express the conditional loops common in practical multi-agent workflows. Another thread of work instead reasons about workflow structure online through speculative execution; e.g., PASTE [11] exploits recurring tool-call patterns and predictable data dependencies to speculatively invoke tools, and Speculative Actions [31] generalizes speculation to agent actions. These methods accommodate dynamic workflows but predict only the next step, which can lead to *myopic eviction* when driving cache eviction; and autoregressive rollout also accumulates errors. Therefore, we develop a multi-step predictor customized for KV-Cache management.

8 Conclusion

We present PBKV for realistic dynamic multi-agent workflows. First, PBKV proposes guidelines for a dedicated *predictor*, i.e., (i) fusing complementary signals from the global call graph and per-request prefill embeddings, and (ii) forecasting over a *multi-step* horizon to avoid myopic decisions. Second, PBKV adopts a *conservative* policy, i.e., (i) hierarchical eviction that combines lifecycle-aware reclamation with lookahead, cross-workflow-aggregated scoring, and (ii) conservative prefetching that consumes only otherwise-idle GPU space and PCIe bandwidth. **Theoretically**, we prove that

PBKV degrades gracefully under prediction error. **Empirically**, PBKV significantly outperforms LRU across models and workloads, and also outperforms the SOTA method on static workflows.

References

- [1] Harrison Chase. LangChain. <https://github.com/langchain-ai/langchain>, 2022. Software.
- [2] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024. URL <https://openreview.net/forum?id=uAjxFFing2>.
- [3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [5] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37: 62557–62583, 2024.
- [6] Yongtong Wu, Shaoyuan Chen, Yinmin Zhong, Rilun Huang, Yixuan Tan, Wentao Zhang, Liyue Zhang, Shangyan Zhou, Yuxuan Liu, Shunfeng Zhou, Mingxing Zhang, Xin Jin, and Panpan Huang. Dualpath: Breaking the storage bandwidth bottleneck in agentic llm inference, 2026. URL <https://arxiv.org/abs/2602.21548>.
- [7] LangChain AI. LangGraph. <https://github.com/langchain-ai/langgraph>, 2024. Software.
- [8] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems journal*, 5(2):78–101, 1966.
- [9] Zaifeng Pan, Ajikumar Patel, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=5Iw1nDtYmT>.
- [10] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- [11] Yifan Sui, Han Zhao, Rui Ma, Zhiyuan He, Hao Wang, Jianxun Li, and Yuqing Yang. Act while thinking: Accelerating llm agents via pattern-aware speculative tool execution. *arXiv preprint arXiv:2603.18897*, 2026.
- [12] Marcel Wagenländer, Otto White, Britannio Jarrett, Pedro Silvestre, Yanda Tao, Guo Li, Huanzhou Zhu, Lluís Vilanova, and Peter Pietzuch. Scepsy: Serving agentic workflows using aggregate llm pipelines, 2026. URL <https://arxiv.org/abs/2604.15186>.
- [13] Jiabin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. Agentic uncertainty quantification. *arXiv preprint arXiv:2601.15703*, 2026.
- [14] Jiabin Zhang, Caiming Xiong, and Chien-Sheng Wu. Agentic confidence calibration. *arXiv preprint arXiv:2601.15778*, 2026.

- [15] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. Agentrx: Diagnosing ai agent failures from execution trajectories. *arXiv preprint arXiv:2602.02475*, 2026.
- [16] Dany Moshkovich, Hadar Mulian, Sergey Zeltyn, Natti Eder, Inna Skarbovsky, and Roy Abitbol. Beyond black-box benchmarking: Observability, analytics, and optimization of agentic systems. *arXiv preprint arXiv:2503.06745*, 2025.
- [17] Dany Moshkovich and Sergey Zeltyn. Taming uncertainty via automation: Observing, analyzing, and optimizing agentic ai systems. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 3840–3844. IEEE, 2025.
- [18] LMSYS Org and SGLang Team. SGLang HiCache: Fast hierarchical KV caching with your favorite storage backends. <https://lmsys.org/blog/2025-09-10-sglang-hicache/>, 9 2025.
- [19] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- [20] Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. Hover: A dataset for many-hop fact extraction and claim verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 3441–3460, 2020.
- [21] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [22] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. Financebench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.
- [23] CrewAI. CrewAI. <https://github.com/crewAIInc/crewAI>, 2023. Software.
- [24] Qiaoling Chen, Zhisheng Ye, Tian Tang, Peng Sun, Boyu Tian, Guoteng Wang, Shenggui Li, Yonggang Wen, Zhenhua Han, and Tianwei Zhang. Concur: High-throughput agentic batch inference of llm via congestion-based concurrency control. *arXiv preprint arXiv:2601.22705*, 2026.
- [25] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, et al. Lmcache: An efficient kv cache layer for enterprise-scale llm inference. *arXiv preprint arXiv:2510.09665*, 2025.
- [26] Hancheng Ye, Zhengqi Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, Ming-Yu Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, and Yiran Chen. KVCOMM: Online cross-context KV-cache communication for efficient LLM-based multi-agent systems. In *The Thirtieth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=yG0ytgjurF>.
- [27] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*, pages 94–109, 2025.
- [28] Hanchen Li, Qiuyang Mang, Runyuan He, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph E. Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. In *ICLR 2026 Workshop on Lifelong Agents: Learning, Aligning, Evolving*, 2026. URL <https://openreview.net/forum?id=sVzK0LC9pn>.
- [29] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, 2024.

- [30] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025. doi: 10.1145/3676641.3716278. URL <https://doi.org/10.1145/3676641.3716278>.
- [31] Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. Speculative actions: A lossless framework for faster AI agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=POG0k5wslg>.
- [32] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=SJU4ayYgl>.
- [33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [34] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [35] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European semantic web conference*, pages 593–607. Springer, 2018.
- [36] Rana Shahout, eran malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. DON’t STOP ME NOW: EMBEDDING BASED SCHEDULING FOR LLMS. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=7JhGdZvW4T>.
- [37] Oscar SKEAN, Md Rifat Arefin, Dan Zhao, Niket Nikul Patel, Jalal Naghiyev, Yann Lecun, and Ravid Shwartz-Ziv. Layer by layer: Uncovering hidden representations in language models. In *International Conference on Machine Learning*, pages 55854–55875. PMLR, 2025.
- [38] Tom Ulanovski, Eyal Blyachman, and Maya Bechler-Speicher. Improving llm predictions via inter-layer structural encoders. In *ICLR Workshop on Geometry-grounded Representation Learning and Generative Modeling*, 2026.
- [39] Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. *Journal of the ACM (JACM)*, 68(4):1–25, 2021.
- [40] Dhruv Rohatgi. Near-optimal bounds for online caching with machine learned advice. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1834–1845. SIAM, 2020.
- [41] Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions. *Communications of the ACM*, 65(7):33–35, 2022.

A Main Results with Standard Deviations.

In our main experiments, we evaluate LRU, KVFlow (only on the static workload due to its inherent limitation), and our PBKV along with its variants on two LLMs and three workloads. Each setting is repeated for 10 runs, with mean and standard deviation reported in Table 2.

B Predictor Details: Architecture and Training

This appendix complements Section 4.1 with the full design of the predictor, including the rationale behind the GraphSAGE backbone, the detailed architecture, and the training procedure.

Overview. The predictor adopts *GraphSAGE* [19] as its backbone, fuses *multiple complementary* signals, and jointly forecasts the next *few* invoked agents (i.e., Multi-Step Prediction).

Table 2: Full results with standard deviations across multi-agent workloads. Each cell reports mean $\pm$ standard deviation over multiple runs. The *static* workload (FinanceBench+CrewAI) is included for comparison with KVFlow, which assumes a predefined static call graph. Concurrency limits are 72 for HoVer+LangChain, 24 for SWE-bench+AutoGen, and 48 for FinanceBench+CrewAI.

Model	Workload	Policy	Latency (s) $\downarrow$	Per-Agent TTFT (s) $\downarrow$	Cache Hit Rate (%) $\uparrow$
Qwen3-32B	HoVer + LangChain	LRU	189.66 $\pm$ 4.70	16.65 $\pm$ 0.78	27.09 $\pm$ 1.16
		PBKV-LAE	146.67 $\pm$ 6.73	11.83 $\pm$ 0.74	44.91 $\pm$ 2.20
		PBKV-HE	108.86 $\pm$ 6.93	8.95 $\pm$ 0.74	66.01 $\pm$ 2.79
		Full PBKV	102.60 $\pm$ 7.28	8.22 $\pm$ 0.71	69.10 $\pm$ 2.23
	SWE-bench + AutoGen	LRU	271.90 $\pm$ 9.14	5.04 $\pm$ 0.26	46.34 $\pm$ 2.01
		PBKV-LAE	233.86 $\pm$ 12.70	3.73 $\pm$ 0.44	59.51 $\pm$ 2.92
		PBKV-HE	177.22 $\pm$ 10.36	2.46 $\pm$ 0.37	75.31 $\pm$ 2.57
		Full PBKV	160.18 $\pm$ 10.89	2.27 $\pm$ 0.35	79.94 $\pm$ 3.26
	FinanceBench + CrewAI (<i>static</i>)	LRU	130.34 $\pm$ 5.37	12.93 $\pm$ 0.51	27.95 $\pm$ 0.82
		KVFlow	101.57 $\pm$ 5.27	9.91 $\pm$ 0.57	39.87 $\pm$ 1.43
		Full PBKV	80.53 $\pm$ 5.06	8.00 $\pm$ 0.48	53.44 $\pm$ 2.07
Qwen3-14B	HoVer + LangChain	LRU	139.59 $\pm$ 3.13	10.90 $\pm$ 0.35	28.79 $\pm$ 1.04
		PBKV-LAE	119.13 $\pm$ 6.04	9.57 $\pm$ 0.58	41.18 $\pm$ 3.17
		PBKV-HE	80.62 $\pm$ 5.13	6.53 $\pm$ 0.34	64.15 $\pm$ 1.99
		Full PBKV	76.15 $\pm$ 5.95	5.50 $\pm$ 0.42	68.29 $\pm$ 2.32
	SWE-bench + AutoGen	LRU	214.74 $\pm$ 6.98	4.18 $\pm$ 0.28	48.53 $\pm$ 3.02
		PBKV-LAE	177.50 $\pm$ 9.05	3.26 $\pm$ 0.51	60.41 $\pm$ 4.62
		PBKV-HE	141.53 $\pm$ 7.57	2.19 $\pm$ 0.43	73.36 $\pm$ 3.95
		Full PBKV	118.61 $\pm$ 8.06	2.05 $\pm$ 0.47	77.07 $\pm$ 3.71
	FinanceBench + CrewAI (<i>static</i>)	LRU	96.11 $\pm$ 4.88	9.02 $\pm$ 0.44	26.58 $\pm$ 0.86
		KVFlow	80.13 $\pm$ 4.16	7.42 $\pm$ 0.41	39.65 $\pm$ 1.84
		Full PBKV	65.01 $\pm$ 4.12	6.06 $\pm$ 0.40	55.07 $\pm$ 2.39

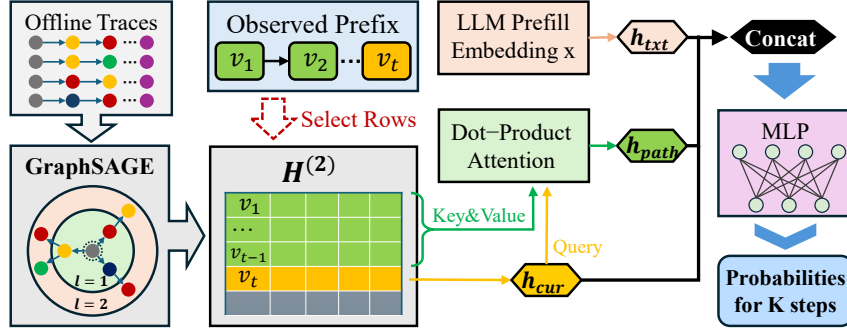


Figure 7: **Architecture of the predictor.** It fuses a topology-aware agent embedding from GraphSAGE (h_{cur}), an attention-based workflow prefix summary (h_{path}), and a semantic signal reused from prefill (h_{txt}), then jointly predicts the next K agent probability distributions via an MLP head.

Why GraphSAGE? Our predictor design follows two desiderata. *First*, the predictor must serve *all* possible workflows across every admissible agent transition, so it is desirable to leverage the structural priors encoded in the global call graph G . *Second*, the dynamic nature of agentic workflows requires the predictor to generalize to unseen subgraphs encountered at runtime.

Guided by these desiderata, we adopt **GraphSAGE** as the backbone of predictor. *First*, GraphSAGE learns node representations directly on the graph through neighborhood sampling and aggregation, thereby preserving the structural priors encoded in agent transitions, which an MLP would otherwise discard. *Second*, GraphSAGE is an *inductive* framework, which performs forward inference on unseen workflow prefix without retraining on the entire graph. This property aligns naturally with the dynamic nature of agentic workflows, and is preferable to transductive alternatives (e.g., GCN [32]) that rely on a fixed training graph. For completeness, we also implement and evaluate several baseline predictors (including MLPs, GCNs, and Markov models), detailed results are in Appendix C.

Architecture. Figure 7 illustrates the architecture of predictor. To forecast the future invocations of an active workflow, the predictor fuses three complementary streams as described below:

(i) *Topology-aware agent representation.* We assign each agent $v \in V$ a learnable embedding $\mathbf{e}_v \in \mathbb{R}^d$, stacked into $\mathbf{H}^{(0)} \in \mathbb{R}^{|V| \times d}$. $A \in \mathbb{R}^{|V| \times |V|}$ denotes the row-normalized forward *transition matrix* estimated from the offline traces. We then apply two GraphSAGE propagation layers:

$$\mathbf{H}^{(\ell)} = \text{ReLU}\left(\left[\mathbf{H}^{(\ell-1)} \parallel A\mathbf{H}^{(\ell-1)}\right](\mathbf{W}^{(\ell)})^\top\right), \quad \mathbf{W}^{(\ell)} \in \mathbb{R}^{d \times 2d}, \quad \ell = 1, 2 \quad (3)$$

where $\parallel$ denotes feature-wise concatenation. Thus, for each agent, the resulting $\mathbf{H}^{(2)}$ encodes both its own identity (through $\mathbf{H}$) and the structural priors induced by its neighborhood in G (through $A\mathbf{H}$).

(ii) *Attention-based history aggregation.* While $\mathbf{H}^{(2)}$ equips each agent with a topology-aware representation, it does not yet capture the state of a *workflow*. In fact, two workflows residing at the same agent v_t may have arrived through very different prefixes. To leverage such prefix information, we summarize the prefix via scaled *dot-product attention*, using the current agent representation $\mathbf{h}_{\text{cur}} = \mathbf{H}_{v_t}^{(2)}$ as the query and the prefix representations as both keys and values:

$$\alpha_i = \frac{\exp((\mathbf{W}_q \mathbf{h}_{\text{cur}})^\top \mathbf{H}_{v_i}^{(2)} / \sqrt{d})}{\sum_{j < t} \exp((\mathbf{W}_q \mathbf{h}_{\text{cur}})^\top \mathbf{H}_{v_j}^{(2)} / \sqrt{d})}, \quad \mathbf{h}_{\text{path}} = \sum_{i < t} \alpha_i \mathbf{H}_{v_i}^{(2)} \quad (4)$$

Compared with naive mean pooling, attention allows the predictor to upweight those prefix agents that are most informative of the current state.

(iii) *Semantic signal from prefill.* Graph topology and workflow history alone do not distinguish requests that share the same prefix but diverge in intent. To capture such *request-specific* semantics, we reuse the post-norm hidden state $\mathbf{x}$ of the last prefill token and project it through a single linear layer, i.e., $\mathbf{h}_{\text{txt}} = \text{ReLU}(\mathbf{W}_t \mathbf{x})$. As $\mathbf{x}$ is a by-product of prefill, this signal is obtained essentially for free.

Multi-step prediction head. The above streams are concatenated and fed into a two-layer MLP with dropout, which jointly emits logits over agents for each of the next K steps. Emitting all K logits from a shared backbone in one forward pass further avoids autoregressive error accumulation and bounds per-invocation latency to that of a single inference.

Training Strategy. We train the predictor on offline invocation traces. For each position t within a workflow, we pair the prefix $(v_1, \dots, v_t)$ and its prefill embedding with the future agents $(v_{t+1}, \dots, v_{t+K})$ as labels. The token $\langle \text{END} \rangle$ is treated as a regular target so that the termination probability $p_{w, \langle \text{END} \rangle}$ is properly learned, and only the padding positions after $\langle \text{END} \rangle$ in a target window are masked from the cross-entropy loss. The resulting predictor has roughly 350K parameters (orders of magnitude smaller than the served LLM) and processes a batch of 1,024 requests in 1.56 ms, rendering its runtime overhead negligible. Detailed hyperparameter settings and training configurations are provided in the supplementary code.

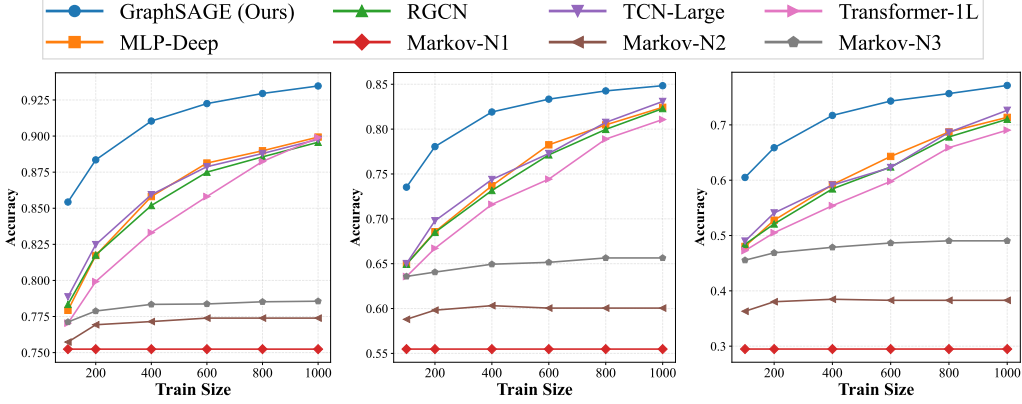
C Workflow Prediction

This appendix supplements Section 4.1 with three additional results: (i) the scaling behavior of our predictor across training-set sizes (Section C.1), (ii) a comparison against alternative predictor families (Section C.2), and (iii) the sensitivity of prediction accuracy to which transformer layer the prefill semantic signal is extracted from (Section C.3).

Setup. All experiments use the HoVer [20] dataset with the LangChain [1] agent framework. We report top-1 accuracy at horizons $k=1, 2, 3$ (denoted s_1, s_2, s_3), averaged over 10 random seeds.

C.1 Scaling Behavior

Figure 8 plots top-1 accuracy against the number of training traces. Two observations emerge. First, learning-based predictors scale with data while the Markov baseline saturates within 100 traces, justifying predicting from a learned representation rather than from tabulated transition probabilities. Second, our predictor leads every learning-based baseline at every horizon and the lead widens as the horizon grows.



(a) Prediction Accuracy of Step 1. (b) Prediction Accuracy of Step 2. (c) Prediction Accuracy of Step 3.

Figure 8: Top-1 prediction accuracy as a function of training-set size at horizons $k=1, 2, 3$. Our predictor leads every baseline at every train size shown.

Table 3: Predictor comparison at 1000 training traces, grouped by architectural family (mean $\pm$ std over 10 seeds).

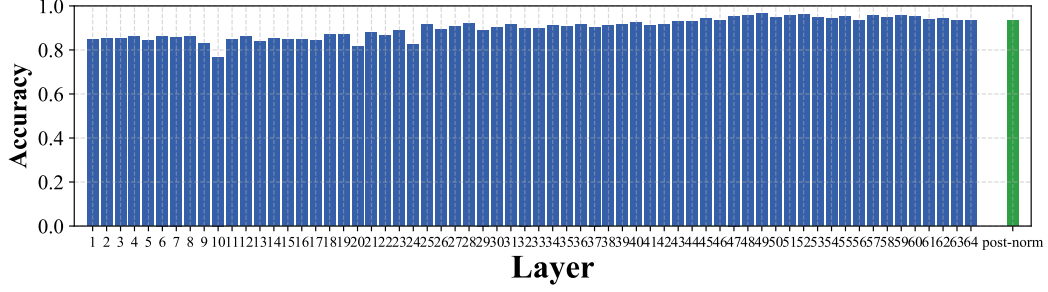
Family	Predictor	s_1	s_2	s_3
GraphSAGE (Ours)		0.935 ± 0.007	0.848 ± 0.012	0.771 ± 0.019
R-GCN	R-GCN	0.896 ± 0.008	0.823 ± 0.011	0.710 ± 0.018
Transformer	1L	0.899 ± 0.008	0.811 ± 0.023	0.690 ± 0.034
	2L	0.896 ± 0.011	0.810 ± 0.022	0.688 ± 0.040
TCN	SMALL	0.901 ± 0.006	0.824 ± 0.015	0.717 ± 0.019
	LARGE	0.898 ± 0.006	0.831 ± 0.007	0.726 ± 0.019
MLP	TINY	0.786 ± 0.003	0.656 ± 0.004	0.494 ± 0.003
	SMALL	0.889 ± 0.004	0.801 ± 0.012	0.684 ± 0.021
	DEEP	0.899 ± 0.006	0.824 ± 0.016	0.714 ± 0.024
Linear Probe	POST-NORM	0.840 ± 0.026	0.732 ± 0.026	0.576 ± 0.036
kNN	$k=5$	0.839 ± 0.008	0.699 ± 0.007	0.564 ± 0.008
	$k=20$	0.817 ± 0.005	0.707 ± 0.007	0.576 ± 0.006
Markov	$n=1$	0.752 ± 0.000	0.555 ± 0.000	0.295 ± 0.000
	$n=2$	0.774 ± 0.000	0.601 ± 0.000	0.383 ± 0.000
	$n=3$	0.786 ± 0.000	0.656 ± 0.000	0.490 ± 0.000

C.2 Comparison Across Predictor Families

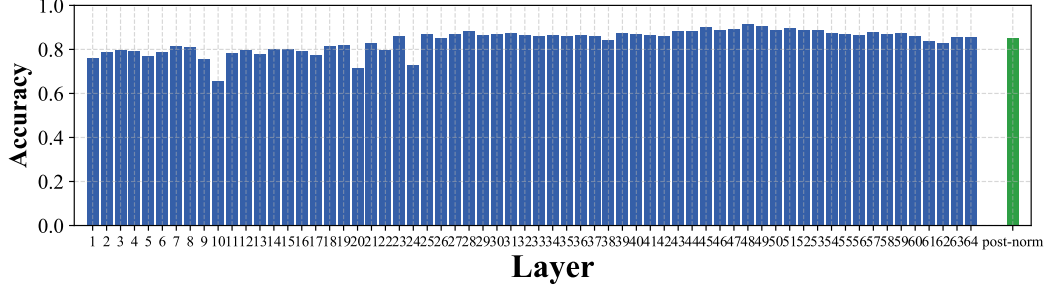
Table 3 compares our predictor against every baseline family we implemented: parameter-free baselines (Markov, kNN), a linear probe on prefill states, MLPs, sequence models (Transformer [33], TCN [34]), and a relational graph encoder (R-GCN [35]). Our predictor outperforms the strongest variant of every other family on all three horizons. The lead widens further at longer horizons, consistent with the multi-step argument that long-horizon prediction benefits most from a rich summary of the workflow prefix.

C.3 Choice of the Layer for Semantic Extraction

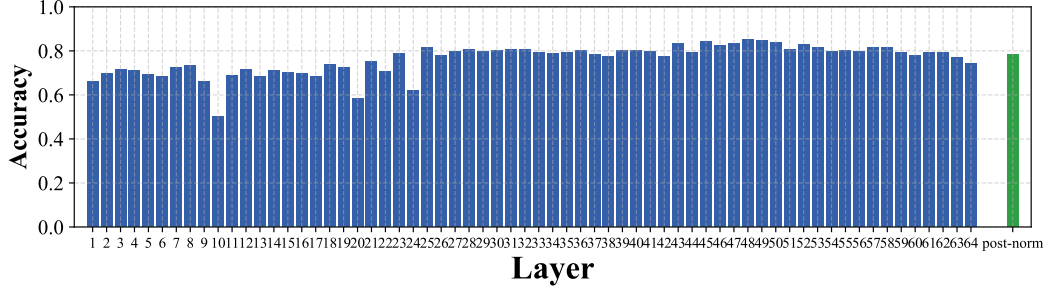
The prefill semantic signal h_{txt} described in Section 4.1 is extracted from a single layer of the served LLM, and a natural question is which layer to choose. Prior studies [36–38] report that the optimal layer for hidden state extraction is uncertain and depends jointly on the served model and the downstream task. We therefore study this choice empirically on Qwen3-32B by training the predictor



(a) Prediction Accuracy of Step 1.



(b) Prediction Accuracy of Step 2.



(c) Prediction Accuracy of Step 3.

Figure 9: Top-1 prediction accuracy as a function of the layer from which the prefill semantic signal h_{txt} is extracted. The served LLM is Qwen3-32B, which exposes 64 transformer block outputs ($\ell=1, \dots, 64$) followed by the post-norm hidden state *post-norm* that is fed into the output head.

with h_{txt} taken from each transformer block output ($\ell=1, \dots, 64$) and from the post-norm hidden state *post-norm*, holding the rest of the architecture and the training protocol fixed.

Figure 9 reports the resulting s_1, s_2, s_3 accuracies. Consistent with prior observations, the optimal layer varies across horizons and is highly specific to the served LLM and the workload at hand: deploying our system on a different LLM or workload would require re-running the layer scan to identify a new optimum. To preserve compatibility with arbitrary served LLMs and workloads, and to keep the predictor self-contained, we extract h_{txt} from *post-norm* in all other experiments. The post-norm hidden state is the standard input to the LLM’s output head, and is therefore universally accessible across LLMs without extra instrumentation, regardless of how many transformer blocks the model contains.

D Sensitivity to the Confidence Decay Coefficient

The score computation uses a coefficient γ to discount the contribution of farther-step predictions, reflecting their decreasing reliability. We study its sensitivity on Qwen3-32B under the HoVer+LangChain workload. As shown in Figure 10, $\gamma = 0.7$ achieves the best performance. A

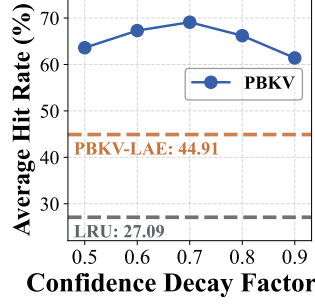


Figure 10: Average cache hit rate vs. confidence decay coefficient γ .

smaller γ can cause the score to decay too rapidly, suppressing lookahead and degrading toward myopic behavior; while a larger γ will over-weight unreliable predictions at distant steps, leading to misguided eviction decisions.

E Aggressive vs. Conservative Prefetching under Varying Prediction Accuracy

E.1 Motivation and Setup

Section 4.3 argues that PBKV adopts a conservative prefetching principle: under dynamic workflows, the uncertainty of prediction makes “displacing known-valuable active cache in exchange for speculatively valuable cache” an asymmetric trade-off between deterministic cost and probabilistic benefit. This appendix empirically validates this design choice.

Aggressive variant. We construct an aggressive prefetching variant that relaxes only the space constraint of Section 4.3, while keeping the rest of the design unchanged (i.e., the predictor, the scoring function, the activation only on pure-decode batches, and the PCIe bandwidth budget S_{bw}). Specifically, beyond the conservative budget S_a , the aggressive variant is additionally permitted to displace at most $\rho \cdot S_{\text{total}}$ of active cache to make room for prefetching, where S_{total} denotes the total GPU cache capacity. Active nodes are evicted in ascending order of their lookahead score, ensuring that the lowest-scored nodes are displaced first.

Prediction noise injection. To isolate the effect of prediction accuracy, we apply a controlled perturbation to the predictor’s output. Let $P_w^{(k)}$ denote the GraphSAGE next-step distribution for workflow w at horizon k , and let U denote the uniform distribution over agents. The perturbed distribution is:

$$\tilde{P}_w^{(k)} = (1 - \lambda) P_w^{(k)} + \lambda U, \quad (5)$$

where $\lambda \in [0, 1]$ denotes the noise level. $\lambda = 0$ corresponds to the original predictor, while $\lambda = 1$ degenerates to a uniform (i.e., random) prediction. The perturbation is applied to the predictor outputs consumed by both eviction and prefetching, simulating a system-wide degradation of the prediction signal.

Workload and metrics. We follow the main experimental setup, i.e., the HoVer + LangChain workload with Qwen3-32B at a concurrency of 72. Each configuration is repeated 10 times, and we report the mean and standard deviation of the average cache hit rate.

E.2 Experiment 1: Sensitivity to the Prefetching Space Budget

We first sweep ρ under the noise-free setting $\lambda = 0$ to identify the optimal aggressiveness of the variant.

As shown in Figure 11, the hit rate of aggressive prefetching first increases and then decreases with ρ , peaking at $\rho = 20\%$ with roughly a 3-percentage-point margin over conservative prefetching. As ρ further grows, the hit rate falls below the conservative baseline, indicating that the cumulative cost of mistakenly displacing active cache outweighs the gain from prefetching.

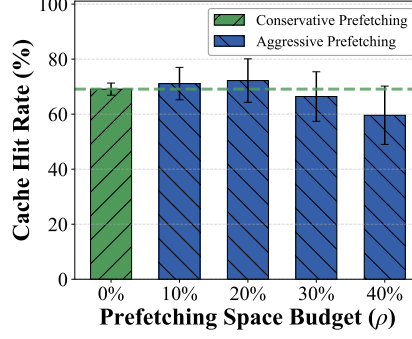


Figure 11: Cache hit rate of conservative and aggressive prefetching across different prefetching space budgets ρ , under the noise-free setting ($\lambda = 0$). Error bars denote standard deviation over 10 trials.

More notably, the standard deviation grows monotonically with ρ even under the noise-free setting. At the optimal point $\rho^* = 20\%$, the standard deviation is already $\sim 3.5\times$ that of conservative prefetching. This indicates that, although aggressive prefetching can deliver expected gains, it harms the stability of the system.

In the following analysis, we use $\rho^* = 20\%$, at which aggressive prefetching achieves its best performance.

E.3 Experiment 2: Sweeping the Prediction Noise

Under $\rho = \rho^*$, we sweep the noise level and compare PBKV-HE (no-prefetching reference), conservative prefetching (the default in PBKV), and aggressive prefetching.

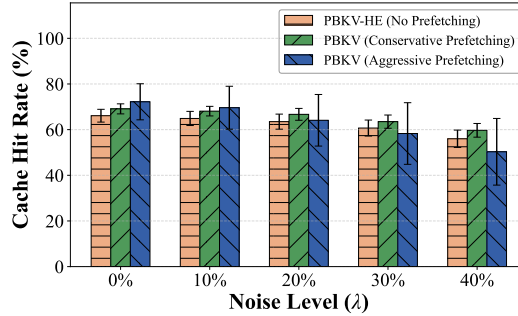


Figure 12: Cache hit rate of PBKV-HE, conservative prefetching, and aggressive prefetching across different noise levels λ . Error bars denote standard deviation over 10 trials.

As shown in Figure 12, two key observations emerge:

(i) The advantageous regime of aggressive prefetching is narrow. Only at $\lambda = 0$ and $\lambda = 10\%$ does aggressive prefetching outperform conservative prefetching, and the margin shrinks rapidly as the noise increases. At $\lambda = 20\%$, aggressive prefetching is already overtaken by conservative prefetching. In practice, the realistic operating regime of aggressive prefetching is even narrower.

(ii) Aggressive prefetching falls below PBKV-HE under moderate-to-high noise. At $\lambda = 30\%$, aggressive prefetching already underperforms PBKV-HE, which performs no prefetching at all; at $\lambda = 40\%$ the gap further widens. This shows that aggressive prefetching becomes counterproductive under inaccurate predictions, with its cost outweighing its benefit. In contrast, conservative prefetching consistently outperforms PBKV-HE across all noise levels, reflecting its lower-bound guarantee.

E.4 Justification of the Design Choice

In summary, aggressive prefetching only achieves a marginal and unstable advantage over conservative prefetching when the prediction accuracy is sufficiently high (i.e., the noise level is below 20%). However, dynamic multi-agent workflows are inherently hard to predict, especially under complex agent frameworks. Therefore, PBKV adopts conservative prefetching to obtain stable gains.

F Scheduler Overhead Analysis

A natural concern with prediction-driven cache management is whether the additional scheduler logic erodes the cache-hit-rate gains. We instrument every PBKV scheduler hook in our SGLang implementation and measure per-call wall-clock latency over a full run of the HoVer+LangChain workload on Qwen3-32B at a concurrency of 72, identical to the operating point used throughout Section 6.2.

Table 4: Per-call latency of PBKV’s scheduler components on Qwen3-32B with the HoVer+LangChain workload at concurrency 72. EvictDecision selects victim nodes on memory pressure; PredictInfer runs one forward pass of the GraphSAGE predictor; ScoreUpdate refreshes a cache node’s heap entry after a new prediction; PrefetchDecision ranks host-resident candidates under the (S_a, S_{bw}) budget; DecodeStep is one batched decode iteration, included as a reference. **Note the unit difference:** ScoreUpdate is in microseconds (μ s); all other components in milliseconds (ms).

Component	Mean	P99
EvictDecision	0.44 ms	0.81 ms
PredictInfer	1.18 ms	1.68 ms
ScoreUpdate	1.53 μ s	2.69 μ s
PrefetchDecision	0.64 ms	1.60 ms
DecodeStep	12.34 ms	43.14 ms

Table 4 reports the mean and P99 latency of EvictDecision, PredictInfer, ScoreUpdate, and PrefetchDecision, with DecodeStep included as a reference for relative cost.

Among them, EvictDecision, ScoreUpdate, and PrefetchDecision run on the CPU scheduler thread and operate purely on PBKV’s heap-based bookkeeping structures, concurrent with the GPU decode kernel. PredictInfer executes on the GPU but is issued on a separate CUDA stream that overlaps with the ongoing decode kernel. None of the four therefore sit on the GPU critical path. ScoreUpdate in particular is essentially free at 1.53 μ s on average, precisely what allows per-node scores to be refreshed at high frequency without serializing the scheduler. Eviction and prefetch decisions both average well under 1 ms (0.44 ms and 0.64 ms), and even predictor inference – the dominant component – averages just 1.18 ms, less than 10% of a mean decode step (12.34 ms), and the other components fall well below this. These overheads do not show up in end-to-end performance: PBKV achieves a $1.85\times$ speedup over LRU on this workload (Section 6.2).

G Smoothness Analysis

Overview. This appendix establishes the smoothness pillar of PBKV’s algorithms-with-predictions guarantee. We organize the analysis into four layers. Lemma G.1 identifies the per-node score $\text{SCORE}(c)$ as the expected discounted miss count attributable to evicting c , thereby grounding the proxy cost in a system-relevant quantity. Lemma G.2 shows that this score is Lipschitz-continuous in the predicted distributions. Corollary G.3 translates this score-level continuity into pairwise ranking stability. Theorem G.4 further lifts it to a cost-level regret bound. All Lipschitz-type results share a single multiplier with a K -independent upper bound $1/(2(1 - \gamma))$ that is also independent of $|\mathcal{W}_{\text{act}}|$ and the cache size, which justifies treating the predictor as a pluggable module: any future improvement in predictor accuracy directly tightens all bounds.

G.1 Setup

Probabilistic model. We posit a ground-truth conditional distribution under which each active workflow w produces, at every step k , a true distribution $P_w^{(k)} \in \Delta^{|V|}$ over $V \cup \{\langle \text{END} \rangle\}$, conditioned on the workflow having not terminated before step k . Here $\Delta^{|V|}$ denotes the $|V|$ -dimensional probability simplex over the $|V| + 1$ outcomes in $V \cup \{\langle \text{END} \rangle\}$, following the standard topological convention. The predictor outputs an estimate $\hat{P}_w^{(k)}$ on the same simplex. Consistent with the sequential agent execution adopted by mainstream multi-agent frameworks (e.g., LangChain [1], Microsoft AutoGen [2], MetaGPT [3]), we model each active workflow as invoking exactly one agent per step (or terminating), so that $P_w^{(k)}$ is the marginal distribution of the step- k invocation conditioned on survival to step k . Let $p_{w, \langle \text{END} \rangle}^{(j)}$ denote the conditional termination probability at step j . The cumulative survival factor follows the standard hazard-product decomposition $s_w^{(k)} = \prod_{j=1}^{k-1} (1 - p_{w, \langle \text{END} \rangle}^{(j)})$. Throughout, $\mathcal{W}_{\text{act}}(c)$ denotes the set of active workflows associated with node c at the moment the eviction decision is made.

Eviction-only abstraction. Following the classical formulation of caching analysis pioneered by Belady [8] and adopted by recent algorithms-with-predictions work [39, 40], we evaluate eviction policies under the *eviction-only abstraction*, i.e., a node, once evicted, remains out of the cache for the entire K -step horizon over which the cost is measured. This abstraction isolates the quality of the eviction decision from orthogonal storage-hierarchy mechanisms (e.g., HiCache re-admission on miss), which are properties of the underlying system rather than of the eviction policy itself. PBKV’s eviction policy operates on the same abstraction and is agnostic to whether a second-tier storage is present. The composition of our analysis with HiCache is discussed at the end of Section G.4.

Access-indicator semantics. For each active workflow $w \in \mathcal{W}_{\text{act}}(c)$, the access indicator $A_w(c) \in \{0, 1\}^{|V|}$ marks the set of agents within w that access c , denoted $\mathcal{O}_w(c) := \{a \in V : A_w(c)_a = 1\}$. Two cases arise. (i) For private cache, $|\mathcal{W}_{\text{act}}(c)|$ is small and $A_w(c)$ is typically one-hot, since the owning agent of c is determined by the upstream agent that produced it within w . (ii) For global cache (e.g., system prompts or tool/agent descriptions), $|\mathcal{W}_{\text{act}}(c)|$ may be large and each $A_w(c)$ may have multiple ones, reflecting that distinct agents within the same workflow all access the shared prefix. The score formula in Eq. (2) treats both cases under a single mathematical form, with the cross-workflow outer sum protecting popular-prefix nodes by design (Section 4.2). For notational convenience, we adopt the convention that $A_w(c)$ is implicitly zero-padded on the $\langle \text{END} \rangle$ coordinate, so that the inner product

$$A_w(c)^\top P_w^{(k)} = \sum_{a \in V} A_w(c)_a P_w^{(k)}(a) = P_w^{(k)}(\mathcal{O}_w(c)) \quad (6)$$

is dimensionally consistent and equals the probability that workflow w invokes some agent in $\mathcal{O}_w(c)$ at step k .

Past access equals future hit. The radix-tree prefix structure adopted by SGLang [5] ensures that $A_w(c)$, defined from the observed access history, also characterizes future hits within the same workflow. Specifically, a cache node c corresponds to a fixed token prefix; if agent a has invoked the radix path through c in w , then any subsequent invocation of a within w (e.g., under a retry loop) prepends the same system prompt and upstream context, and therefore traverses the same prefix and hits c provided c remains cached. Conversely, an agent that has never accessed c does not have c on its execution path and will not hit c in future invocations within w . Hence, throughout the analysis, “ w would hit c at step k if c remained cached” is equivalent to “ w invokes some $a \in \mathcal{O}_w(c)$ at step k ”.

Unit-size abstraction and the role of node size. A natural question is why $\text{SCORE}(c)$ in Eq. (2) does not contain an explicit $|c|$ factor. This is a deliberate design choice: for a cache node c of size $|c|$, both the space cost of retaining it and the wall-clock cost of a miss on it scale linearly with $|c|$, since a miss triggers the re-prefill (or PCIe transfer under HiCache) of $|c|$ tokens. The two factors of $|c|$ therefore cancel out when the quantity of interest is value per unit space. By construction, $\text{SCORE}(c)$ measures exactly this per-unit-space value, which is also the canonical ranking criterion of fractional-knapsack-style cache replacement: ranking by per-unit-space value, rather than by absolute value, is what minimizes total miss cost under a space budget. Multiplying $\text{SCORE}(c)$ by $|c|$ would conflate “valuable” with “large” and bias eviction against small-but-hot nodes, which is the opposite of the intended behavior.

For the regret analysis, we adopt the standard unit-size abstraction [8, 39, 40], under which each node contributes an equal volume to the eviction budget and the budget- B eviction reduces to selecting B lowest-scoring nodes. This abstraction is consistent with the size-heterogeneous setting via a standard reduction in which each node of size $|c|$ is treated as $|c|$ unit-size virtual nodes sharing the same per-unit-space score $\text{SCORE}(c)$, whereby cardinality-constrained selection over virtual nodes recovers size-constrained selection over original nodes. Since radix-tree nodes are page-sized in practice and $|c| \ll B$, the boundary effect of integer-budget rounding is at most one node and does not affect the asymptotic behavior of the bound. The Lipschitz and regret bounds below therefore transfer to the size-heterogeneous setting without modification of the multiplier $1/(2(1 - \gamma))$.

Perturbation model. Following standard practice in algorithms-with-predictions analyses for caching [39, 40], we measure prediction error via the per-step ℓ_1 deviation and its node-local discount-weighted aggregate:

$$\epsilon_w^{(k)} := \|P_w^{(k)} - \widehat{P}_w^{(k)}\|_1, \quad \epsilon_c^\gamma := \sum_{w \in \mathcal{W}_{\text{act}}(c)} \sum_{k=1}^K \gamma^{k-1} \epsilon_w^{(k)}. \quad (7)$$

Throughout, we let $\text{SCORE}(c)$ and $\widehat{\text{SCORE}}(c)$ denote the per-node scores computed from $\{P_w^{(k)}\}$ and $\{\widehat{P}_w^{(k)}\}$ respectively, both via Eq. (2).

G.2 From Score to Expected Miss Count

We first establish the system-level meaning of $\text{SCORE}(c)$, which grounds the subsequent regret analysis in a quantity of direct interest to caching theory.

Lemma G.1 (Score equals expected discounted miss count attributable to eviction). *Under the eviction-only abstraction (Section G.1), for any cache node c that is evicted at the decision point and not re-admitted within the horizon,*

$$\text{EMC}(c) := \sum_{k=1}^K \gamma^{k-1} \mathbb{E}[\text{misses on } c \text{ at step } k] = \text{SCORE}(c), \quad (8)$$

where the expectation is taken under the ground-truth distribution and the misses are aggregated across all $w \in \mathcal{W}_{\text{act}}(c)$.

Proof. Fix a cache node c and a workflow $w \in \mathcal{W}_{\text{act}}(c)$. Let $\#\text{miss}_w^{(k)}(c) \in \{0, 1\}$ denote the indicator that w incurs a miss on c at step k . Since w invokes exactly one agent per step (or terminates), and any invocation of an agent $a \in \mathcal{O}_w(c)$ would have traversed c by the radix-tree prefix structure (Section G.1), we have

$$\#\text{miss}_w^{(k)}(c) = \mathbf{1}[w \text{ survives to step } k \text{ and invokes some } a \in \mathcal{O}_w(c)], \quad (9)$$

where the equality uses that, under the eviction-only abstraction, every such invocation is a miss. The events $\{w \text{ invokes } a \text{ at step } k\}_{a \in V}$ are mutually exclusive conditional on survival to step k , so

$$\Pr[w \text{ invokes some } a \in \mathcal{O}_w(c) \mid w \text{ survives to } k] = \sum_{a \in \mathcal{O}_w(c)} P_w^{(k)}(a) = A_w(c)^\top P_w^{(k)}. \quad (10)$$

Multiplying by $\Pr[w \text{ survives to } k] = s_w^{(k)}$ gives $\mathbb{E}[\#\text{miss}_w^{(k)}(c)] = s_w^{(k)} \cdot A_w(c)^\top P_w^{(k)}$. Summing across $w \in \mathcal{W}_{\text{act}}(c)$ by linearity of expectation and discounting by γ^{k-1} over $k = 1, \dots, K$, comparing with Eq. (2) yields $\text{EMC}(c) = \text{SCORE}(c)$. $\square$

Implications. Lemma G.1 provides three benefits. (i) It removes any circularity between the scoring rule and the cost it is evaluated against, since $\text{SCORE}(c)$ is now identified with a quantity defined purely from the ground-truth distribution and the cache state. (ii) It places PBKV’s analysis on the same footing as classical caching results [8], where eviction policies are evaluated through their effect on miss counts, independent of any specific reload mechanism. (iii) It motivates the proxy cost adopted in Section G.4 as a system-relevant rather than ad-hoc surrogate.

G.3 Lipschitz Continuity of the Score

We next show that $\text{SCORE}(c)$ depends continuously on the predicted distributions, with a Lipschitz constant whose dependence on the prediction horizon K admits a uniform K -independent upper bound.

Lemma G.2 (Lipschitz continuity of the score). *For any cache node c and any pair of distribution sets $\{P_w^{(k)}\}$ and $\{\widehat{P}_w^{(k)}\}$ in $\Delta^{|V|}$,*

$$\left| \text{SCORE}(c) - \widehat{\text{SCORE}}(c) \right| \leq \frac{1 - \gamma^K}{2(1 - \gamma)} \epsilon_c^\gamma \leq \frac{\epsilon_c^\gamma}{2(1 - \gamma)}. \quad (11)$$

The Lipschitz multiplier admits a K -independent upper bound $1/(2(1 - \gamma))$ that is also independent of $|\mathcal{W}_{\text{act}}(c)|$ and the cache size; the dependence on these quantities is fully isolated in ϵ_c^γ .

Proof. We bound the per-term deviation $|s_w^{(k)} A_w(c)^\top P_w^{(k)} - \widehat{s}_w^{(k)} A_w(c)^\top \widehat{P}_w^{(k)}|$ in two steps and then aggregate over k and w .

Step 1 (per-term bound). The survival factor $s_w^{(k)}$ takes values in $[0, 1]$ as a product of $[0, 1]$ terms, and so does $\widehat{s}_w^{(k)}$. The standard telescoping identity for products of $[0, 1]$ -valued sequences gives

$$|s_w^{(k)} - \widehat{s}_w^{(k)}| \leq \sum_{j=1}^{k-1} |p_{w, \langle \text{END} \rangle}^{(j)} - \widehat{p}_{w, \langle \text{END} \rangle}^{(j)}|. \quad (12)$$

For any pair of probability distributions P, Q on a finite outcome space and any subset S of outcomes, the difference of subset-event probabilities is bounded by the total variation distance, $|P(S) - Q(S)| \leq \text{TV}(P, Q) = \frac{1}{2} \|P - Q\|_1$. Applied to the singleton $S = \{\langle \text{END} \rangle\}$, this yields $|p_{w, \langle \text{END} \rangle}^{(j)} - \widehat{p}_{w, \langle \text{END} \rangle}^{(j)}| \leq \frac{1}{2} \epsilon_w^{(j)}$, and substituting into (12) gives

$$|s_w^{(k)} - \widehat{s}_w^{(k)}| \leq \frac{1}{2} \sum_{j=1}^{k-1} \epsilon_w^{(j)}. \quad (13)$$

Similarly, by the zero-padding convention (6), $A_w(c)^\top P_w^{(k)} = P_w^{(k)}(\mathcal{O}_w(c)) \in [0, 1]$ is a subset-event probability under $P_w^{(k)}$, and the same holds for $A_w(c)^\top \widehat{P}_w^{(k)}$. Applying the same total-variation bound to the subset $S = \mathcal{O}_w(c)$,

$$|A_w(c)^\top P_w^{(k)} - A_w(c)^\top \widehat{P}_w^{(k)}| \leq \frac{1}{2} \epsilon_w^{(k)}. \quad (14)$$

Combining (13) and (14) via the elementary identity $|uv - \widehat{u}\widehat{v}| \leq |v - \widehat{v}| + |u - \widehat{u}|$ for $u, \widehat{u}, v, \widehat{v} \in [0, 1]$ yields

$$\left| s_w^{(k)} A_w(c)^\top P_w^{(k)} - \widehat{s}_w^{(k)} A_w(c)^\top \widehat{P}_w^{(k)} \right| \leq \frac{1}{2} \sum_{j=1}^k \epsilon_w^{(j)}. \quad (15)$$

Step 2 (aggregation). Multiplying (15) by γ^{k-1} , summing over $k = 1, \dots, K$, and exchanging the order of summation between k and j ,

$$\sum_{k=1}^K \gamma^{k-1} \cdot \frac{1}{2} \sum_{j=1}^k \epsilon_w^{(j)} = \frac{1}{2} \sum_{j=1}^K \gamma^{j-1} \epsilon_w^{(j)} \cdot \frac{1 - \gamma^{K-j+1}}{1 - \gamma} \leq \frac{1 - \gamma^K}{2(1 - \gamma)} \sum_{j=1}^K \gamma^{j-1} \epsilon_w^{(j)}, \quad (16)$$

where the inner sum is a finite geometric series and the inequality uses $1 - \gamma^{K-j+1} \leq 1 - \gamma^K$ for $j \geq 1$ and $\gamma \in (0, 1)$. Summing across $w \in \mathcal{W}_{\text{act}}(c)$ yields the tighter form $(1 - \gamma^K) \epsilon_c^\gamma / (2(1 - \gamma))$ in Eq. (11), and the looser corollary $\epsilon_c^\gamma / (2(1 - \gamma))$ follows from $1 - \gamma^K \leq 1$. $\square$

Locality remark. Only workflows in $\mathcal{W}_{\text{act}}(c)$ contribute to ϵ_c^γ , so prediction errors on workflows w with $A_w(c) = 0$ leave $\text{SCORE}(c)$ unchanged. This locality reflects the radix-tree prefix structure respected by the scoring rule and matches the design intent of Section 4.2.

Corollary G.3 (Ranking stability). *Let c_1, c_2 be two cache nodes with $\text{SCORE}(c_1) > \text{SCORE}(c_2)$, and let $\Delta := \text{SCORE}(c_1) - \text{SCORE}(c_2)$. If*

$$\frac{1 - \gamma^K}{2(1 - \gamma)} (\epsilon_{c_1}^\gamma + \epsilon_{c_2}^\gamma) < \Delta, \quad (17)$$

then $\widehat{\text{SCORE}}(c_1) > \widehat{\text{SCORE}}(c_2)$, and the eviction preference between c_1 and c_2 is preserved.

Proof. By Lemma G.2 applied to c_1 and c_2 separately, together with the triangle inequality. $\square$

G.4 Eviction Cost Regret

We now lift the score-level continuity to a cost-level regret bound, which is the form prescribed by the smoothness desideratum of the algorithms-with-predictions framework [41].

Proxy cost. We adopt

$$\mathcal{L}(E) := \sum_{c \in E} \text{SCORE}(c), \quad (18)$$

which, by Lemma G.1, equals the expected discounted miss count incurred by evicting the set E under the eviction-only abstraction. This grounds the proxy cost in the canonical quantity studied since Belady [8] and isolates the eviction-policy quality from orthogonal storage-hierarchy concerns. The same surrogate has also been used in algorithms-with-predictions analyses for caching [39, 40], where costs are likewise defined through predicted access patterns rather than realized latencies.

Regret definition. For the regret analysis, we specialize $\{P_w^{(k)}\}$ to the ground-truth distributions and $\{\widehat{P}_w^{(k)}\}$ to the predictor outputs. Freeing a budget of B nodes reduces to a cardinality-constrained selection (Section G.1). PBKV ranks candidate nodes by $\widehat{\text{SCORE}}$ and evicts

$$\widehat{E}_B := \arg \min_{|E|=B} \sum_{c \in E} \widehat{\text{SCORE}}(c), \quad (19)$$

whereas the cost-minimizing eviction set under the ground truth is $E_B^* := \arg \min_{|E|=B} \mathcal{L}(E)$. The eviction cost regret of PBKV is

$$\mathcal{R}(B) := \mathcal{L}(\widehat{E}_B) - \mathcal{L}(E_B^*) \geq 0, \quad (20)$$

where non-negativity follows from the optimality of E_B^* for $\mathcal{L}$.

Theorem G.4 (Eviction cost regret bound). *Under the eviction-only abstraction and the unit-size abstraction (Section G.1), for any predictor outputs $\{\widehat{P}_w^{(k)}\}$, the eviction cost regret of PBKV satisfies*

$$\mathcal{R}(B) \leq \sum_{c \in \widehat{E}_B \Delta E_B^*} \delta_c \leq \frac{1 - \gamma^K}{2(1 - \gamma)} \sum_{c \in \widehat{E}_B \Delta E_B^*} \epsilon_c^\gamma, \quad (21)$$

where Δ denotes the symmetric set difference and $\delta_c := |\text{SCORE}(c) - \widehat{\text{SCORE}}(c)|$. As the prediction error vanishes, $\mathcal{R}(B) \rightarrow 0$.

Proof. Partition $\widehat{E}_B \cup E_B^*$ into $\mathcal{A} := \widehat{E}_B \setminus E_B^*$, $\mathcal{B} := E_B^* \setminus \widehat{E}_B$, and $\mathcal{C} := \widehat{E}_B \cap E_B^*$. Both eviction sets have cardinality B , hence $|\mathcal{A}| = |\mathcal{B}|$. By construction of $\widehat{E}_B$ as the minimizer of $\sum_{c \in E} \widehat{\text{SCORE}}(c)$ over cardinality- B sets, and after cancelling the common $\mathcal{C}$ -terms,

$$\sum_{c \in \mathcal{A}} \widehat{\text{SCORE}}(c) \leq \sum_{c \in \mathcal{B}} \widehat{\text{SCORE}}(c). \quad (22)$$

Therefore

$$\mathcal{R}(B) = \sum_{c \in \mathcal{A}} \text{SCORE}(c) - \sum_{c \in \mathcal{B}} \text{SCORE}(c) \quad (23)$$

$$\leq \sum_{c \in \mathcal{A}} (\text{SCORE}(c) - \widehat{\text{SCORE}}(c)) + \sum_{c \in \mathcal{B}} (\widehat{\text{SCORE}}(c) - \text{SCORE}(c)) \quad (24)$$

$$\leq \sum_{c \in \mathcal{A} \cup \mathcal{B}} \delta_c = \sum_{c \in \widehat{E}_B \Delta E_B^*} \delta_c, \quad (25)$$

where the first inequality follows from (22) and the second from $|\text{SCORE}(c) - \widehat{\text{SCORE}}(c)| \leq \delta_c$ on each side. The remaining inequality of Eq. (21) follows by applying Lemma G.2 to each δ_c . $\square$

Interpretation. Theorem G.4 bounds the regret of PBKV’s eviction decisions on the expected discounted miss count, the canonical cost in caching analysis dating back to Belady [8]. Two observations are worth emphasizing. First, the multiplier $(1 - \gamma^K)/(2(1 - \gamma))$ matches that of Lemma G.2, so the score-level and cost-level bounds share a single K -independent upper bound $1/(2(1 - \gamma))$. Second, only nodes in the symmetric difference $\widehat{E}_B \triangle E_B^*$ contribute to $\mathcal{R}(B)$. A correctly ranked node, relative to the eviction boundary, contributes zero regret regardless of the magnitude of its individual score error. This boundary-localized regret structure indicates that the bound is governed by ranking errors near the eviction frontier rather than by aggregate score errors, consistent with the ranking-stability view of Corollary G.3.

G.5 Summary

This appendix establishes the smoothness pillar of PBKV’s algorithms-with-predictions guarantee along four layers: identification of $\text{SCORE}(c)$ as the expected discounted miss count under the eviction-only abstraction (Lemma G.1), per-node score Lipschitz continuity (Lemma G.2), pairwise ranking stability (Corollary G.3), and cost-level regret on the resulting proxy cost (Theorem G.4). All Lipschitz-type results share a single K -independent multiplier $1/(2(1 - \gamma))$, independent of $|\mathcal{W}_{\text{act}}|$ and the cache size. By analyzing eviction quality under the same eviction-only abstraction adopted in classical caching theory, the analysis treats PBKV’s eviction policy as a self-contained component decoupled from any specific storage-hierarchy mechanism. This justifies treating the predictor as a pluggable module in PBKV’s design (Section 4.2): any future improvement in predictor accuracy directly tightens all bounds, and the bounds themselves remain meaningful in deployments without second-tier storage.